



Bahria University
Discovering Knowledge

ROBOTIC SYSTEM & PROGRAMMING LAB

OPEN ENDED LAB TASK

CLASS: BS RIS 5-A

FALL-2025

OEL Title

Turtle Speed Monitoring Service + Motion Control

GROUP MEMBER'S LIST

MEMBER'S NAME	ENROLLMENT NO
MALAHIMA ADEEL	02-239232-043
UROOJ KHAN	02-239232-052
SHAYAN ALI	02-239232-068

SUBMITTED TO: SIR HAMZA

Objectives:

- ❖ One turtle moves in a pattern.
- ❖ Implement a service that returns (x, y).
- ❖ Client calls the service repeatedly to compute speed and direction.
- ❖ If speed > 3.0, client sends cmd_vel to slow it down.
- ❖ Show service-topic integration.

1) Introduction and Background:

In this project, we simulated a turtle moving on the screen using **Turtlesim**. The goal was to monitor the turtle's motion and control its speed in real-time.

We implemented a **ROS service** that provides the turtle's current position (x, y). A client continuously requests this service to calculate the turtle's **speed and direction**.

If the turtle's speed exceeds a threshold (3.0), the client publishes a **cmd_vel** message. This demonstrates how **ROS services and topics** can be integrated for real-time motion control.

2) Methodology and Key Steps:

We followed these steps:

I. Turtle Motion Node:

- The turtle moves in a circular pattern with slight variations in speed to make motion more natural.
- Speed is varied using a sine function.
- Maximum speed is capped to prevent excessive velocity.

II. Service Server Node:

- Subscribes to /turtle1/pose to get live turtle coordinates.
- Implements a service /get_turtle_state that returns the turtle's current (x, y).

III. Client Node:

- Repeatedly calls the service to get the latest position.
- Calculates **speed and direction** between consecutive positions.
- Logs all data to the terminal for monitoring.
- If speed exceeds 3.0, publishes a **cmd_vel** message.
- Ensures warnings are triggered only once per speed crossing.

3) Key Implementation Steps:

I. ROS & Turtlesim were used to simulate the environment.

II. Python scripts implemented three main nodes:

- `turtle_motion.py` → moves the turtle in a patterned motion.
- `position_service_server.py` → service server providing (x, y) when requested by a client.
- `speed_monitor_client.py` → client calculating speed and sending commands to slow down if necessary.

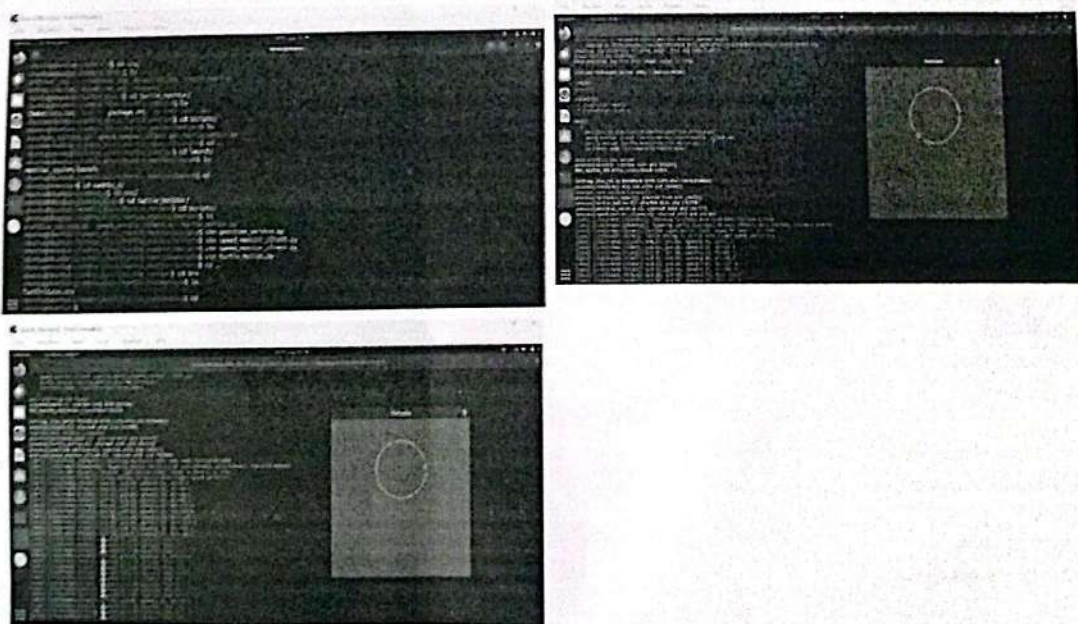
III. ROS Concepts Used:

- `rospy.Publisher` → for sending velocity commands (`cmd_vel`).
- `rospy.Service` & `rospy.ServiceProxy` → for service communication.
- `/turtle1/pose` topic → provides live turtle positions.
- Speed and direction calculations:
 - Distance = $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
 - Speed = distance / time
 - Direction = $\text{atan2}(dy, dx)$

IV. Safety Measures:

- Maximum speed capped at 4.0.
- Small random variations to make motion realistic.
- Speed warnings only triggered once per threshold crossing.

4) Simulation Screenshots:



70



1



10

1

position_service_server.py

$$T = 1/50 \quad f = \frac{1}{T}$$

```
if __name__ == '__main__':
    rospy.init_node('position_service_server')
    rospy.Subscriber('/turtle1/pose', Pose, pose_callback)
    service = rospy.Service('/get_turtle_state', TurtleState, handle_state_request)
    rospy.loginfo("Service '/get_turtle_state' ready to send x, y, time.")
    rospy.spin()
```

CLIENT NODE:

```
#!/usr/bin/env python3
```

```
import rospy
```

```
import math
```

```
from geometry_msgs.msg import Twist
```

```
from turtle_monitor.srv import TurtleState
```

```
class SpeedMonitor:
```

```
    def __init__(self):
```

```
        rospy.init_node('speed_monitor_client', anonymous=True)
```

```
        self.pub = rospy.Publisher('/turtle1/cmd_vel', Twist, queue_size=10)
```

```
        rospy.wait_for_service('/get_turtle_state')
```

```
        self.service = rospy.ServiceProxy('/get_turtle_state', TurtleState)
```

```
        self.prev_x = None
```

```
        self.prev_y = None
```

```
        self.prev_t = None
```

```
        self.rate = rospy.Rate(2) # 2 Hz
```

```
        self.warning_triggered = False
```

```
    def run(self):
```

```
        while not rospy.is_shutdown():
```

```
            res = self.service()
```

```
            x, y = res.x, res.y
```

```
            t = rospy.Time.now().to_sec()
```

```
            if self.prev_x is not None:
```

```
                dx = x - self.prev_x
```

```
                dy = y - self.prev_y
```

```
                dt = t - self.prev_t
```

```
                if dt < 0.05:
```

```
                    self.rate.sleep()
```

```
                    continue
```

```
                distance = math.sqrt(dx**2 + dy**2)
```

```
                speed = distance / dt
```

```
                direction = math.degrees(math.atan2(dy, dx))
```

```
                rospy.loginfo(f"x={x:.2f}, y={y:.2f}, speed={speed:.2f}, dir={direction:.1f}")
```

```
            # Warning and slow-down only once per crossing above 3
```

```
            if speed > 3.0 and not self.warning_triggered:
```

```
                rospy.logwarn("□ Speed %.2f exceeded 3! Slowing down..." % speed)
```

```
                twist = Twist()
```

```
                twist.linear.x = 2.5
```

```
                twist.angular.z = direction * math.pi / 180 # convert deg to rad
```

```

        self.pub.publish(twist)
        self.warning_triggered = True
    elif speed <= 3.0:
        self.warning_triggered = False

    self.prev_x, self.prev_y, self.prev_t = x, y, t
    self.rate.sleep()

if __name__ == '__main__':
    try:
        SpeedMonitor().run()
    except rospy.ROSInterruptException:
        pass

```

TURTLE MOTION NODE:

turtle_motion.py

```

#!/usr/bin/env python3
import rospy
from geometry_msgs.msg import Twist
import math
import time
import random

def move_turtle():
    rospy.init_node('turtle_motion', anonymous=True)
    pub = rospy.Publisher('/turtle1/cmd_vel', Twist, queue_size=10)
    rate = rospy.Rate(50) # 50 Hz

    radius = 2.0
    start_time = time.time()
    warning_triggered = False

    rospy.loginfo("Turtle moving in circle with slight speed variations")

    while not rospy.is_shutdown():
        elapsed = time.time() - start_time

        # Base linear speed mostly between 1.5-3.0
        speed = 2.0 + 1.0 * math.sin(0.2 * elapsed)

        # Small random variation to make motion natural
        speed += random.uniform(-0.2, 0.2)

        # Cap speed between 0.5-4
        speed = max(0.5, min(speed, 4.0))

        # Warning only once per crossing above 3
        if speed > 3.0 and not warning_triggered:
            rospy.logwarn("⚠ Speed is %.2f (greater than 3!) % speed)
            warning_triggered = True
        elif speed <= 3.0:
            warning_triggered = False

        # Angular speed to maintain circle

```

```
angular_speed = speed / radius
```

```
vel_msg = Twist()
```

```
vel_msg.linear.x = speed
```

```
vel_msg.angular.z = angular_speed
```

```
pub.publish(vel_msg)
```

```
rate.sleep()
```

```
if __name__ == '__main__':
```

```
    try:
```

```
        move_turtle()
```

```
    except rospy.ROSInterruptException:
```

```
        pass
```

5) Conclusion:

We made a smart turtle system that checks its own speed and controls it. The turtle moves in a circle, and the client keeps watching it. If it goes too fast, the client slows it down. This project helped us learn about ROS services, topics, and how to control motion using code.